

Control Tower — Prototype Architecture

Trimmed to exactly what's needed to demo the five core features. Nothing else.

Scope

This is a working-prototype architecture, not a production one. Everything here exists to make one of the five features actually run and be visible in a demo. Auth, migrations, caching, message queues, and file-per-entity organization are deliberately left out — they add structure for a codebase that will live for years, not for a system you'll build once and demo once. If you want any of that added back later, ask and it's a small addition, not a rewrite.

How each feature maps to files

Feature	Backend	Frontend
Geospatial Dashboard	routes.py (GeoJSON from /facilities)	MapView.jsx, statusColors.js
Inventory Triage	triage.py, models.py	TriageSummary.jsx
Escalation Hierarchy	escalation.py, routes.py (/transfers, /transfers/{id}/respond)	TransferPanel.jsx
ML Forecasting	forecast_model.py, forecasting.py, routes.py (/forecast, /forecast/suggested-quantity)	ForecastChart.jsx
Logistics / ETA Tracking	escalation.py (distance calc)	TransferPanel.jsx (shows km + ETA)

Backend (FastAPI)

```
backend/
|-- main.py          # FastAPI app, CORS, mounts routes.py
|-- database.py      # SQLAlchemy engine/session (SQLite for this
|                    # prototype; schema.sql stays the documented
|                    # Postgres+PostGIS target design, not what
|                    # actually runs)
|-- models.py        # All SQLAlchemy models in one file, matching
|                    # schema.sql's tables (SQLite-adapted -- see
|                    # the file's own header for the two differences)
|-- schemas.py       # All Pydantic request/response models, one file
|-- routes.py        # All endpoints: /facilities, /inventory,
|                    # /forecast, /forecast/suggested-quantity,
|                    # /transfers, /transfers/{id}/respond
|-- triage.py        # classify_status() -> surplus/warning/critical,
|                    # applied to live facility_inventory rows
|-- forecasting.py   # NEW -- ML-suggested request quantity: sums
|                    # forecast_model.py's predicted consumption
|                    # over a medicine's lead time, minus current
|                    # stock
|-- escalation.py    # single-target match (nearest peer, else walk
|                    # parent_facility_id up the hierarchy); dire
|                    # situations fan out to multiple targets at
|                    # once; partial fulfillment spawns a follow-up
|                    # request for the remainder
|-- forecast_model.py # already built - 7-day XGBoost forecast
|-- generate_mock_data.py # already built - simulated facilities + outbreak
|-- schema.sql       # already built - documented Postgres/PostGIS
|                    # schema (reference design, not what runs)
`-- requirements.txt
```

Why flat: models.py and schemas.py are each one file instead of one-per-entity. Splitting them into a folder just adds import overhead for no benefit at this scale. Same logic for routes.py — one file with several route groups is easier to navigate during a hackathon than jumping across an endpoints/ folder.

Services stay separate from routes: `triage.py`, `forecasting.py`, and `escalation.py` hold the actual logic. `routes.py` just calls them. This is the one separation worth keeping — it means you can test "does escalation pick the right facility" without running the API at all.

Frontend (React + Vite)

```
frontend/
|-- src/
|   |-- components/
|   |   |-- MapView.jsx           # Leaflet map, color-coded facility markers
|   |   |-- TriageSummary.jsx     # counts: X critical, Y warning, Z surplus
|   |   |-- ForecastChart.jsx    # 7-day line chart with confidence band
|   |   |-- TransferPanel.jsx    # request form (ML-suggested quantity,
|   |                                   # editable), single-target request
|   |                                   # status, and an incoming-requests view
|   |                                   # to accept or reject requests targeting
|   |                                   # this facility
|   |-- api.js                   # one fetch function per backend endpoint
|   |-- App.jsx                  # single dashboard page, renders all 4
|   |-- main.jsx
|-- package.json
`-- vite.config.js
```

Why one page: DashboardPage / FacilityDetailPage / TransferManagementPage as separate routed pages made sense for a product with real navigation. For a demo, one App.jsx that lays out the map, triage counts, forecast chart, and transfer panel together is what a judge actually wants to see in one screen — no clicking through pages to find the feature you're describing.

Deliberately left out (add later if needed)

- Authentication / login
- Database migrations (Alembic) — call database.py's `init_db()` once and move on
- Docker Compose, Redis, background job queues
- WebSocket/SSE live push — polling every few seconds looks identical in a demo
- Separate model/schema files per entity
- Multi-page routing / navigation
- Delivery confirmation endpoint — accepted transfers go to "in_transit"; there's no endpoint yet to mark one "completed". Flag before adding it — not an oversight.

Suggested build order

1. schema.sql + generate_mock_data.py — done
2. forecast_model.py — done
3. database.py + models.py — done
4. API_CONTRACT.md updated for the two new endpoints — done
5. schemas.py — done
6. triage.py — done
7. forecasting.py — done
8. escalation.py
9. routes.py + main.py
10. Frontend: api.js → MapView → TriageSummary → ForecastChart → TransferPanel → App.jsx